LOT DE TRAVAIL 18

Protocole appareil et squelette de firmware

Jalon H3 (préparation) | terminé sur PC ; jamais exécuté sur une carte | 2026-11-09

Résumé

Un squelette de firmware pour l'ESP32-C3 construit les trames exactement comme le protocole écrit, vérifiées octet par octet contre l'encodeur du simulateur, avec un tampon circulaire hors ligne qui n'écrase jamais des données non synchronisées. **Rien n'a été exécuté sur une vraie carte** : les composants ne sont pas commandés (sources en §6).

1. Contexte et problématique

Le projet dépend d'un appareil réel, mais le matériel n'est pas encore acheté. Pour ne pas perdre de temps, le code qui peut être prouvé sur un PC (formats, mémoire tampon) est écrit et testé maintenant ; les étapes qui exigent le matériel restent une liste de contrôle.

2. Objectifs

- Des trames identiques à celles du simulateur et du protocole (`docs/protocol/device-protocol.md`).
- Une mémoire tampon hors ligne sans perte (règle « ne jamais perdre une séance »).
- Aucun identifiant Wi-Fi ni jeton dans le code source.

3. Démarche et méthode

Tests natifs sur PC compilés avec le compilateur `zig` trouvé sur la machine (pas de chaîne PlatformIO nécessaire pour la partie testée). Une première livraison a été rejetée par la revue parce qu'elle ne produisait que l'en-tête de 18 octets : la seconde emballe aussi les échantillons.

4. Conception

Format brut : en-tête (magie `DSFV`, version, fréquences, nombres d'échantillons) puis échantillons ADXL355 et LIS2DW12 sur 16 bits en petit-boutiste, et pression en centi-kPa. Le tampon circulaire refuse l'écriture plutôt que d'écraser un élément non synchronisé.

5. Réalisation

Dossier `firmware/esp32c3-dsfav/` : bibliothèque de trames, tampon, tests natifs, ébauches d'acquisition (ADXL355, LIS2DW12, capteur de pression, client WebSocket). Liste de mise en route `docs/hardware/bringup.md` pour les étapes H0 à H3.

6. Vérification et résultats

VÉRIFICATION	RÉSULTAT	SOURCE
Trame complète identique à l'encodeur du simulateur	oui	`node test/run-native.mjs`, étiquette `[G1]`
Tampon : refus d'écraser un élément non synchronisé	oui	étiquette `[NO-LOSS]`
Aucun identifiant ou jeton dans `src` et `lib`	oui	contrôle par recherche dans le même exécuteur
Exécution sur une carte	**non réalisée**	matériel non commandé

7. Décisions et justifications

- Les identifiants Wi-Fi viennent d'une phase de mise en service, jamais du code (règle 6 de `AGENTS.md` §5).
- Le compilateur local sert aux tests ; la compilation pour la carte est une étape distincte à valider avec la chaîne PlatformIO.

8. Limites et points ouverts

Le bruit réel des capteurs, l'étalonnage de la pression, la consommation, la portée Wi-Fi et la tenue du tampon sur la mémoire flash réelle ne sont pas testés. La taille de la flash de la carte reste une question ouverte qui fixe la taille du tampon. L'ébauche de la carte n'est pas encore branchée sur la bibliothèque de trames.

9. Ce qu'il faut retenir

- On prouve sur PC tout ce qui ne dépend pas du matériel, et on dit clairement le reste.
- Une trame se vérifie contre une autre implémentation, pas contre elle-même.

- La mémoire tampon est une exigence de sécurité des données, pas un détail.

---

10. Questions possibles du jury

- **Le firmware fonctionne-t-il ?** Ses formats et son tampon sont prouvés sur PC ; son exécution sur carte est la prochaine étape.
- **Que se passe-t-il quand le Wi-Fi tombe ?** Les séances restent dans le tampon et sont envoyées plus tard, une seule fois grâce à la clé d'idempotence.
- **Pourquoi ne pas commander le matériel d'abord ?** Cela reste nécessaire ; ce travail réduit simplement le risque le jour où les pièces arrivent.

Références

`firmware/esp32c3-dsfav/` · `docs/protocol/device-protocol.md` · `docs/hardware/bringup.md` · `scripts/simulator/`